



REACT

RP // INTERVIEW SERIES

0 – 3 YEARS

VOL 03 · FRONTEND INTERVIEW

React Interview Roadmap.

0 – 3 Years Experience

Senior-style answers tuned for the 0–3 years experience window — hooks, internals, performance, and the tricky follow-ups asked at product companies.

React 18/19

Hooks

Reconciliation

Performance

Suspense

RSC

React Query

120+

PAGES

30+

QUESTIONS

5.4 MB

PDF SIZE

AUTHORED BY

Rutik Pimpale

UPDATED

May 2026

NAVIGATION

Table of Contents

CLICK ANY ENTRY TO JUMP TO THE SECTION

01 TOPIC 1 — REACT FUNDAMENTALS

- Q1: What is JSX and how does the Virtual DOM work?
- Q2: Functional vs Class Components, and how do props work?
- Q3: What's the difference between state and props?
- Q4: Controlled vs Uncontrolled Components

02 TOPIC 2 — CORE HOOKS

- Q1: useState — pitfalls and batching
- Q2: useEffect — dependency array deep dive
- Q3: useEffect cleanup & race conditions
- Q4: useRef — when to reach for it

03 TOPIC 3 — PERFORMANCE HOOKS

- Q1: useMemo vs useCallback
- Q2: React.memo — when does it actually help?
- Q3: Why does useEffect run twice in development?

04 TOPIC 4 — ADVANCED HOOKS

- Q1: useContext — and the provider pattern
- Q2: useReducer vs useState
- Q3: Custom hooks — rules and patterns

05 TOPIC 5 — REACT INTERNALS

- Q1: Reconciliation and the role of keys
- Q2: Fiber — what changed and why it matters
- Q3: Why is setState async and how does batching work?

06 TOPIC 6 — STATE MANAGEMENT

- Q1: Context vs Redux vs Zustand — when to use which
- Q2: React Query / TanStack Query — server state done right

07 TOPIC 7 — REACT 18 / 19 FEATURES

- Q1: Concurrent rendering & useTransition
- Q2: Suspense for data fetching
- Q3: React Server Components (RSC)

08 TOPIC 8 — FORMS & ROUTING

- Q1: React Hook Form — why everyone uses it
- Q2: React Router v6 — loaders & actions

09 TOPIC 9 — TESTING & PATTERNS

- Q1: React Testing Library — guiding principles
- Q2: HOC vs Render Props vs Compound Components

10 TOPIC 10 — REAL-WORLD & TRICKY

- Q1: Why should keys NOT be the array index?
- Q2: Memory leaks in useEffect
- Q3: Error Boundaries — why they exist
- Q4: Lifting state up vs colocation

11 CLOSING

TOPIC 1 — REACT FUNDAMENTALS

Q1: What is JSX and how does the Virtual DOM work?

A: JSX is **syntactic sugar over** `React.createElement(...)`. The browser doesn't understand JSX — Babel/SWC compiles every JSX tag down to a function call that returns a plain JS object describing the UI. That object tree is the **Virtual DOM**.

When state changes, React builds a **new VDOM tree**, **diffs it against the previous one**, and applies the **smallest set of real DOM mutations** needed. Real DOM updates are expensive; comparing JS objects is cheap. That's the entire performance story.

- JSX → `React.createElement('div', { className: 'x' }, 'Hello')` → `{ type: 'div', props: {...}, children: [...] }`
- This object is **a description, not the DOM itself**. React decides what real DOM operations to run.
- The "diff" is the **reconciliation algorithm** (covered in Topic 5).
- Calling JSX "HTML in JS." It's not HTML — `class` becomes `className`, `for` becomes `htmlFor`, events are camelCase (`onClick`, not `onlick`), and inline style is an object, not a string.
- Thinking the Virtual DOM is "faster than the DOM." It's not — DOM is faster than VDOM. The VDOM is faster than **the worst-case re-renders you'd write by hand without a framework**.

```
// JSX
const el = <h1 className="title">Hi {name}</h1>;

// What Babel actually emits (React 17+ automatic runtime is similar)
const el = React.createElement('h1', { className: 'title' }, 'Hi ', name);

// Which evaluates to:
// { type: 'h1', props: { className: 'title', children: ['Hi ', name] } }
```

- *Is the Virtual DOM unique to React?* — No. Vue, Preact, Inferno all use one. Svelte and Solid skip it entirely (compile-time reactivity).
- *Why doesn't React update the real DOM directly on every state change?* — Because batching + diffing means **one** DOM commit per state change instead of N, which is what makes 60fps possible.

Q2: Functional vs Class Components, and how do props work?

A: As of 2026, **functional components are the default and class components are legacy**. New code should be functions. But interviewers still ask the comparison because there's nuance.

Functional component — a plain JS function that returns JSX. State and lifecycle come via **hooks**.

```
function Greeting({ name }) {  
  return <h1>Hello {name}</h1>;  
}
```

Class component — extends `React.Component`, uses `this.state`, `this.setState`, and lifecycle methods (`componentDidMount`, etc.).

```
class Greeting extends React.Component {  
  render() { return <h1>Hello {this.props.name}</h1>; }  
}
```

- Hooks let you **share stateful logic** across components without HOC/render-prop ceremony.
- No `this` binding gotchas.
- Better minification, smaller bundles, easier static analysis.
- Concurrent features (Suspense, transitions) work cleanly with functions.
- Props are **read-only** inside the receiving component. Mutating `props.x` is a bug.
- Props flow **one-way**, parent → child. To change parent state from a child, pass a callback prop.
- A new **object/array/function literal** as a prop on every parent render creates a new reference, which can defeat memoization. (Topic 3.)
- Spreading too liberally: `<Input {...props} />` — passes unrelated props down to a DOM node and React warns about "unknown DOM attribute."
- Mutating arrays in props: `props.items.push(x)` — never. Build a new array.
- Forgetting `children` is just a prop: `<Card>hello</Card>` ⇔ `<Card children="hello" />`.
- *Can you still write a class component?* — Yes, fully supported. But you'll hit a wall with newer hooks-only APIs (`useTransition`, `useDeferredValue`, etc.).
- *What's PropTypes / why don't we use it anymore?* — Runtime validation library. Replaced by **TypeScript** in any modern codebase.

Q3: What's the difference between state and props?

A: Both are inputs to a component, but their **ownership** is opposite.

	Props	State
Owned by	Parent component	The component itself
Mutable from inside?	No (read-only)	Yes (via setter)
Triggers re-render?	When parent passes new value	When <code>setState</code> is called
Use for	Configuration, data passed in	Things that change inside this component

The rule of thumb: if a value can be computed from props, it's **derived data** — don't put it in state. Put it in a `const` (or `useMemo` if expensive).

```
// ❌ Bug - `email` won't update if parent passes a new userEmail
function Profile({ userEmail }) {
  const [email, setEmail] = useState(userEmail);
  return <input value={email} onChange={e => setEmail(e.target.value)} />;
}

// ✅ Fix - only do this if you genuinely want a temporary, editable copy
// and need to reset it on prop change:
function Profile({ userEmail }) {
  const [email, setEmail] = useState(userEmail);
  useEffect(() => { setEmail(userEmail); }, [userEmail]);
  // ...
}

// ✅✅ Better - fully controlled by parent
function Profile({ email, onChange }) {
  return <input value={email} onChange={e => onChange(e.target.value)} />;
}
```

- *Where should state live?* — At the **lowest common ancestor** of all components that need to read or write it. (See Topic 10 — lifting state vs colocation.)
- *Why does mutating state directly not trigger re-render?* — React uses `Object.is` to compare prev/next state. If the reference is the same, it bails out.

Q4: Controlled vs Uncontrolled Components

A: It's all about **who owns the form value** — React state, or the DOM itself.

— React state is the source of truth.

```
function Form() {  
  const [name, setName] = useState('');  
  return <input value={name} onChange={e => setName(e.target.value)} />;  
}
```

Uncontrolled — the DOM owns the value; you read it via **ref** when you need it.

```
function Form() {  
  const ref = useRef(null);  
  return (  
    <input ref={ref} defaultValue="" />  
    <button onClick={() => alert(ref.current.value)}>Submit</button>  
  );  
}
```

- **Controlled** — most forms. You want validation as the user types, conditional UI based on input, etc.
- **Uncontrolled** — file inputs (`<input type="file">` is **always** uncontrolled — you can't set its value programmatically), or large forms where you only care about values at submit time. **React Hook Form** uses uncontrolled internally for performance.
- Setting `value=""` (controlled) without an `onChange` handler — input becomes read-only and React warns.
- Switching mid-flight from `defaultValue` to `value` — React errors with "A component is changing an uncontrolled input to be controlled." Pick one and stick.
- Doing controlled input on a 50-field form. Every keystroke re-renders the whole form. Use React Hook Form (uncontrolled) instead.
- *Why is `<input type="file">` always uncontrolled?* — Browser security: scripts can't set a file input's value, only the user can.
- *What's the perf cost of a controlled input on a giant form?* — One re-render per keystroke per field. On 100+ fields with complex children, this is visible jank.

TOPIC 2 — CORE HOOKS

Q1: useState — pitfalls and batching

A: `useState` returns `[value, setter]`. Looks simple, has 3 traps interviewers love.

The setter captures the value from the render in which it was created.

```
function Counter() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(count + 1); // reads `count` from THIS render
    setCount(count + 1); // still THIS render's count → both set to 1, not 2
  }
  return <button onClick={handleClick}>{count}</button>;
}
```

Fix: — gets the latest value from React, ignoring closure.

```
setCount(c => c + 1);
setCount(c => c + 1); // ✅ now goes from 0 → 2
```

If the initial value is expensive, pass a function, not a value.

```
useState(JSON.parse(huge)) // ❌ runs every render
useState(() => JSON.parse(huge)) // ✅ runs only on mount
```

Trap 3 — Object/array state. State updates **don't merge** like `this.setState` did in classes. You replace the whole thing.

```
const [user, setUser] = useState({ name: '', email: '' });
setUser({ name: 'Rutik' }); // ❌ email is gone
setUser(prev => ({ ...prev, name: 'Rutik' })); // ✅
```

Batching (React 18+): all `setState` calls inside the same event handler / effect / promise callback are batched into **one** re-render. Pre-18, only React-event-handler updates were batched — promise callbacks weren't, which caused surprise re-renders.

- Does `setState` always trigger a re-render? — No. If `Object.is(prev, next)` is true, React bails out. So `setCount(count)` is a no-op.
- How do you opt out of batching? — `flushSync(() => setX(...))` from `react-dom`. Rarely needed.

Q2: useEffect — dependency array deep dive

A: `useEffect(fn, deps)` runs `fn` **after** the browser paints, but only when one of `deps` has changed by `Object.is`.

The 3 forms:

- `useEffect(fn)` — runs after **every** render. Almost always a bug.
- `useEffect(fn, [])` — runs **once** after mount. (Twice in StrictMode dev — see Topic 3 Q3.)
- `useEffect(fn, [a, b])` — runs after mount, then any time `a` or `b` changes by reference.

```
function Search({ filters }) {
  useEffect(() => {
    fetch('/api', { body: JSON.stringify(filters) });
  }, [filters]); // 🚩 if parent passes {} as a literal, this re-runs every render
}
```

If the parent does `<Search filters={{ active: true }} />`, that's a **new object on every render** of the parent. Reference inequality → effect re-runs forever → infinite refetch.

Fixes:

1. Depend on **primitive** fields, not the whole object: `[filters.active, filters.q]`.
 2. Memoize the object in the parent: `useMemo(() => ({ active: true }), [active])`.
 3. Move the fetch into a query library (React Query) and pass the deps as a query key — it handles this for you.
- Lying to the linter (`// eslint-disable-next-line react-hooks/exhaustive-deps`) instead of fixing the dep. The linter is right 95% of the time.
 - Using `useEffect` to **transform** props into derived state. Don't — compute it during render or with `useMemo`.
 - Putting `setState` inside an effect with a missing dep → infinite loop.


```
// ❌ infinite loop - setCount triggers re-render → effect re-runs
useEffect(() => { setCount(count + 1); });

// ✅ run once
useEffect(() => { setCount(c => c + 1); }, []);
```

- Why use `Object.is` and not `===`? — `Object.is(NaN, NaN)` is `true`, `NaN === NaN` is `false`. Same for `+0` vs `-0`. Edge cases that matter.
- Does the effect run before or after the browser paints? — **After** for `useEffect`. **Before** for `useLayoutEffect` (which blocks paint — use sparingly, only for measurements).

Q3: useEffect cleanup & race conditions

A: The function you `return` from a `useEffect` is the **cleanup**. It runs:

1. **Before** the next effect runs (when deps change), and
2. **On unmount**.

This is how you avoid memory leaks (subscriptions, timers, listeners) and in async code.

```
function User({ id }) {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch(`/api/users/${id}`).then(r => r.json()).then(setData);
  }, [id]);
}
```

User clicks fast: `id=1`, `id=2`. Two fetches start. If `id=1`'s response **comes back last** (slow network), the UI shows user 1's data even though the user wanted user 2. Bug.

```
useEffect(() => {
  let cancelled = false;
  fetch(`/api/users/${id}`)
    .then(r => r.json())
    .then(d => { if (!cancelled) setData(d); });
  return () => { cancelled = true; };
}, [id]);
```

```

useEffect(() => {
  const ctrl = new AbortController();
  fetch(`/api/users/${id}`, { signal: ctrl.signal })
    .then(r => r.json())
    .then(setData)
    .catch(err => { if (err.name !== 'AbortError') throw err; });
  return () => ctrl.abort();
}, [id]);

```

Best — don't write this code by hand. Use **React Query** / **SWR**. They handle race conditions, caching, retries, deduplication, and stale-while-revalidate for free. (Topic 6 Q2.)

- `clearInterval` / `clearTimeout`
- Event listeners: `el.removeEventListener(...)`
- WebSocket close
- Subscription unsubscribe
- *What happens if I forget the cleanup on a `setInterval`?* — Multiple intervals stack up over renders. Visible as "the counter speeds up the longer the page is open."
- *Does cleanup run on unmount even with empty deps?* — Yes. `[]` means "run once on mount, cleanup on unmount."

Q4: useRef — when to reach for it

A: `useRef` gives you a **mutable container** that **persists across renders** and **does NOT trigger re-renders** when you change its `.current`.

Two distinct uses:

```

function FocusInput() {
  const ref = useRef(null);
  useEffect(() => { ref.current.focus(); }, []);
  return <input ref={ref} />;
}

```

```
function Stopwatch() {
  const intervalId = useRef(null);
  const start = () => {
    intervalId.current = setInterval(tick, 1000);
  };
  const stop = () => clearInterval(intervalId.current);
}
```

The mental model: `useState` for things the **UI depends on**, `useRef` for things the UI **doesn't care about** (timer IDs, previous values, "did this run already" flags).

- Mutating `ref.current` and expecting a re-render. It won't. Use state.
- Using `useRef` for derived values you could just compute on render.
- Forgetting that the ref is `null` on first render — DOM refs only attach **after** mount, so don't read `.current` during render.

```
// ❌ logs null on first render
function Box() {
  const ref = useRef();
  console.log(ref.current); // null on mount
  return <div ref={ref} />;
}

// ✅ read it after mount
useEffect(() => { console.log(ref.current); }, []);
```

- **Forwarding refs?** — `forwardRef((props, ref) => ...)` lets a parent pass a ref to a child's internal DOM node. Replaced by simply accepting `ref` as a prop in React 19.
- **Difference between `useRef` and `createRef`?** — `createRef` returns a **new ref every render** (used in classes). `useRef` returns the **same ref across renders**.

TOPIC 3 — PERFORMANCE HOOKS

Q1: useMemo vs useCallback

A: Both **cache a value across renders** based on dependencies. They differ in **what** they cache.

- `useMemo(fn, deps)` — caches the **return value** of `fn`. Use for **expensive computations** or stable **object/array references**.

- `useCallback(fn, deps)` — caches the **function itself**. Equivalent to `useMemo(() => fn, deps)`. Use when passing a function to a memoized child or to an effect.

```
const sorted = useMemo(() => bigList.sort(cmp), [bigList]); // value
const onClick = useCallback(() => doThing(id), [id]); // function
```

The single most important rule: these hooks have a **cost** (running deps comparison, holding the cached value). They are **not free**.

- **Don't memoize by default.** It's premature optimization.
- Use `useMemo` when the computation is **measurably expensive** (sort/filter on >1k items, complex aggregations).
- Use `useMemo` / `useCallback` to **stabilize a reference** you pass to:
 - `React.memo` 'd children (otherwise memo never hits — see next question).
 - `useEffect` deps (otherwise the effect re-runs every render).
- Skip both for simple JSX or cheap maps. The wrapper costs more than what it "saves."

React Compiler (React 19): the new compiler does this for you automatically. In codebases using it, you stop writing `useMemo` / `useCallback` by hand.

- Memoizing every function in a component "to be safe." Adds noise, slows down renders.
- `useCallback(fn, [])` with deps that should be there → stale closure bug.
- Wrapping a child in `React.memo` while still passing a non-memoized inline object/function as a prop. The memo never matches.
- *Does `useMemo` guarantee the value won't be recomputed?* — No. React **may** discard the cache (low memory, etc.). Treat it as a hint, not a contract.
- *What's the cheaper alternative for "stable dispatch" callbacks?* — `useReducer` (the dispatch fn is always stable) or `useEvent` (proposed, not stable yet — but the React Compiler obsoletes the question).

Q2: React.memo — when does it actually help?

A: `React.memo(Component)` returns a wrapped version that **skips re-rendering if the props are shallow-equal** to the previous render.

It only helps when of these are true:

1. The parent re-renders often.
2. The child renders something **expensive**.

3. The props passed to the child are **stable references** across renders.

```
const Big = React.memo(function Big({ items, onClick }) {
  // expensive
});

function Parent() {
  return <Big items={[1, 2, 3]} onClick={() => doThing()} />;
  // 🚩 [1,2,3] and () => ... are NEW REFERENCES every render
  // → memo NEVER hits → wasted overhead
}
```

Both the array literal and the inline function create a new identity every render. `React.memo`'s shallow comparison sees them as different and re-renders anyway.

```
function Parent() {
  const items = useMemo(() => [1, 2, 3], []);
  const onClick = useCallback(() => doThing(), []);
  return <Big items={items} onClick={onClick} />;
}
```

- Component is cheap (a label, a small button). Comparing props costs more than re-rendering.
- Props change every time the parent re-renders anyway.
- You haven't measured. **Always profile first** with React DevTools Profiler.
- Memoizing **everything** out of habit. Bundle size and code complexity grow; perf doesn't.
- Custom comparator (`React.memo(C, areEqual)`) with bugs. Most of the time, shallow is what you want.
- *How is `React.memo` different from `PureComponent`?* — Same idea (shallow prop comparison). `PureComponent` is for class components.
- *What does the React Compiler do here?* — It auto-memoizes components and values where safe, so you don't write `memo` / `useMemo` / `useCallback` by hand.

Q3: Why does useEffect run twice in development?

A: React Strict Mode. In development only, React wraps your app in `<StrictMode>` which deliberately:

- Mounts → unmounts → re-mounts every component once on first render.
- Calls every effect → its cleanup → the effect again.

It's a for your code. The point is to surface bugs that would only show up later (e.g., when a component is hidden + reshown, or when navigation re-mounts a route).

```
useEffect(() => {  
  socket.connect(); // forgotten cleanup → after strict-mode double-mount,  
                    // you have 2 connections instead of 1  
}, []);
```

If you write the cleanup correctly, the double-mount is a no-op:

```
useEffect(() => {  
  socket.connect();  
  return () => socket.disconnect(); // ✅ now strict mode behaves identical to prod  
}, []);
```

- Trying to "disable" Strict Mode because effects run twice. Don't. Fix the effect.
- Counting in an effect (`count++` in a ref) and being confused by the doubling. Use the cleanup, or move out of the effect.
- Posting `POST /api/track-mount` from a `useEffect(() => fetch('/api/track-mount'), [])` — you'll get **two** events in dev. Either accept it (dev only), or use a `useRef` "did I run" guard, or use a real analytics library that dedupes.
- *Does Strict Mode run twice in production?* — **No**. It's a development-only behavior.
- *Do all hooks run twice?* — Effects do. State initializers and `useMemo` factories may also be invoked twice (deliberately, to catch impurity). Render functions may also run twice.

TOPIC 4 — ADVANCED HOOKS

Q1: useContext — and the provider pattern

A: Context is React's built-in way to share a value with **deeply nested children** without prop-drilling. Every component inside a `<Provider value={...}>` can read `value` via `useContext(MyContext)`.

```

const ThemeContext = createContext('light');

function App() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext.Provider value={theme}>
      <Page />
    </ThemeContext.Provider>
  );
}

function Button() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>Hi</button>;
}

```

```

// ❌ value is a NEW object every render of App → every consumer re-renders every time
<ThemeContext.Provider value={{ theme, setTheme }}>

// ✅ memoize
const ctx = useMemo(() => ({ theme, setTheme }), [theme]);
<ThemeContext.Provider value={ctx}>

```

- Theme, locale, current user, feature flags — values that rarely change but are read **everywhere**.
- Keeping a small piece of UI state (modal open, sidebar collapsed) accessible across a subtree.
- Frequently-changing values (mouse position, form state). Every consumer re-renders. Use Zustand / Jotai / Redux for that.
- Server data. Use React Query — it solves caching, deduping, and refetch for you.
- Treating Context as a state-management library. It's a **transport** for state, not a store.
- One giant **AppContext** with everything in it. Split into focused contexts (UserContext, ThemeContext, ...) so unrelated re-renders don't propagate.
- *Can a consumer skip re-rendering when only part of the context changed?* — Not natively — every consumer re-renders. Workaround: split contexts, or use a selector library (**use-context-selector**).
- *React 19 simplification?* — **<MyContext value={x}>** (no **.Provider**) and **use(MyContext)** work in conditionals.

Q2: useReducer vs useState

A: `useReducer` is `useState` with a **reducer function** in the middle. It's better when:

- State updates depend on **multiple fields** at once.
- You have **many actions** that change state in different ways.
- You want a **stable dispatch** function (the dispatch fn never changes — `useCallback`-free).

```
function reducer(state, action) {
  switch (action.type) {
    case 'add':    return { ...state, items: [...state.items, action.item] };
    case 'remove': return { ...state, items: state.items.filter(i => i.id !== action.id) };
    case 'reset':  return { items: [], total: 0 };
    default:      throw new Error(`unknown ${action.type}`);
  }
}

const [state, dispatch] = useReducer(reducer, { items: [], total: 0 });

dispatch({ type: 'add', item: { id: 1, name: 'x' } });
```

Mental model: `useReducer` ≈ "Redux for one component." Same pattern (action → pure reducer → new state), no library.

- One or two unrelated values.
- Simple toggles / counters / form fields.
- The setter logic isn't shared anywhere else.
- Mutating state inside the reducer. Reducers must be **pure** — return a new object.
- Putting side effects (fetches, navigation) inside the reducer. Side effects belong in event handlers / effects, not reducers.
- *Why does `dispatch` have a stable identity?* — React guarantees it. You can pass `dispatch` to children without `useCallback`.
- *When would you pair `useReducer` with `useContext`?* — When you want global-ish state with predictable updates and don't need a full library. Common in mid-size apps before adopting Zustand/Redux.

Q3: Custom hooks — rules and patterns

A: A custom hook is **just a function that calls other hooks**. By convention, the name starts with **use** so the linter can enforce the rules of hooks.

The Rules of Hooks (these are non-negotiable):

1. Only call hooks at the **top level** of a component or another hook. **Never** in conditionals, loops, or nested functions.
2. Only call hooks from **React functions** (components or other hooks).

Why? React tracks hook calls **by call order**. If you skip one with an **if**, every subsequent hook gets the wrong slot's data → silent corruption.

```
// ❌ BREAKS – order of useState calls changes between renders
function Bad({ enabled }) {
  if (enabled) {
    const [x] = useState(0);
  }
  const [y] = useState(0);
}

// ✅ Always call at top level
function Good({ enabled }) {
  const [x] = useState(0);
  const [y] = useState(0);
  // use `enabled` to decide what to render, not what to call
}
```

```

function useDebounceValue(value, delay = 300) {
  const [debounced, setDebounce] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounce(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}

// Usage
function Search() {
  const [q, setQ] = useState('');
  const dq = useDebounceValue(q, 400);
  useEffect(() => { fetch(`/api?q=${dq}`); }, [dq]);
  return <input value={q} onChange={e => setQ(e.target.value)} />;
}

```

- `useDebounceValue`, `useThrottledCallback`
- `useLocalStorage(key, initial)` — sync state to localStorage
- `useMediaQuery('(min-width: 768px)')`
- `useClickOutside(ref, handler)`
- `usePrevious(value)`
- Putting business logic in a custom hook just to "extract." Extract only when there's **reuse** or genuine separation of concerns.
- A custom hook that does **5 unrelated things**. Split it.
- *Why must hook names start with `use`?* — The ESLint plugin uses that convention to enforce rules of hooks.
- *Can a custom hook have generics in TypeScript?* — Yes. Common pattern:
`useLocalStorage<T>(key, initial: T): [T, Setter<T>]` .

TOPIC 5 — REACT INTERNALS

Q1: Reconciliation and the role of keys

A: Reconciliation is React's algorithm for figuring out **the smallest set of DOM mutations** to take you from the previous VDOM tree to the new one.

1. **Different element types** (`<div>` → `<span>`) → tear down the whole subtree and rebuild.

2. **Same type** → keep the node, just diff props and recurse into children.

For lists, React needs **keys** to match old children to new children. Without a stable key, it falls back to **index-based matching** — which is wrong almost any time the list is reordered, filtered, or items are inserted at the front.

```
// ❌ index-as-key bug
{items.map((item, i) => <Row key={i} data={item} />)}

// User deletes item 0:
//   index 0 → was item A, is now item B → React reuses A's DOM node, just changes props
//   → input/uncontrolled focus state from A leaks into B's row!
```

```
{items.map(item => <Row key={item.id} data={item} />)}
```

The visible bug: focus, scroll position, animation state, and uncontrolled input values get bound to the **DOM node**, not the React element. Wrong keys → wrong values stick to the wrong row.

- The list **never reorders, filters, or inserts in the middle**.
- Pure render output (no inputs, no internal state per row).
- Using `Math.random()` as the key. Now every row is "new" every render → catastrophic perf and lost state.
- Using the array index when you have a stable ID — pure laziness, often a real bug.
- *Why does the key need to be unique only among siblings?* — React only diffs siblings against each other.
- *What's the cost of a wrong key?* — Lost component state (any `useState` inside the row), focus loss, animation glitches, occasional perf regressions.

Q2: Fiber — what changed and why it matters

A: Fiber is the rewrite of React's reconciler that shipped with React 16. It made reconciliation **interruptible**.

Pre-Fiber (stack reconciler): React walked the tree synchronously. A 50ms render **blocked** the main thread for 50ms — input lag, dropped frames.

Fiber: the tree is a **linked list of "fiber" nodes**. React processes them in chunks, **yields back to the browser** between chunks, and resumes later. If a higher-priority update (input typing, animation) comes in mid-render, React can **abort** the current work and start over with the new state.

This is the foundation for:

- **Concurrent rendering** (React 18+)
- **Suspense**
- `useTransition` (mark an update as low-priority)
- `useDeferredValue`

You probably don't need to know the full algorithm — most interviewers test the **why**:

"Why was Fiber needed? Because the synchronous reconciler couldn't be paused, so a heavy render blocked input. Fiber chunks the work and yields, enabling time-slicing and concurrent features."

- Saying "Fiber is faster." It's not — for a single render, it's similar or slightly slower. **It's about responsiveness, not throughput.**
- Saying Fiber "uses fibers from OS threads." No — JavaScript is single-threaded. Fibers here are just **JS objects** representing units of work.
- *Does Fiber make my app multi-threaded?* — No, still single-threaded. Just yields between work units.
- *What is "time slicing"?* — Splitting render work into ~5ms chunks so the browser can interleave painting, scrolling, and input.

Q3: Why is `setState` async and how does batching work?

A: It's not really "async" — it's **batched**. When you call `setState`, React doesn't update immediately. It **schedules** an update, finishes the current event handler / effect / etc., **collects all the `setState` calls**, and then runs **one** re-render with the merged state.

```
function onClick() {  
  setCount(count + 1);  
  console.log(count); // logs the OLD value - no re-render has happened yet  
}
```

`count` doesn't update inside the handler. It updates on the **next render**.

without it, this code would re-render 3 times:

```
function onClick() {  
  setName('a');  
  setEmail('b');  
  setActive(true);  
}
```

With batching: re-render after the handler finishes. Big perf win.

React 18 changed batching scope. Before 18, batching only worked inside React event handlers. In promise callbacks / setTimeout / native events, each `setState` triggered its own re-render. React 18 introduced **automatic batching everywhere**.

```
// React 17 - 2 re-renders, React 18 - 1 re-render  
fetch('/x').then(() => {  
  setName('a');  
  setEmail('b');  
});
```

Force-flush batching: `flushSync(() => setX(...))` from `react-dom`. Useful when you need the DOM updated *before* the next line (e.g., focusing an element that just appeared).

- Reading state right after `setState` and being surprised it didn't change. Use the new value directly, or read it in a `useEffect`.
- Calling `setState` in a loop with closure values. Use the functional updater: `setX(x => ...)`.
- *Can I opt out of automatic batching in React 18?* — Yes, with `flushSync`. Don't unless you know why.
- *What's the difference between "batching" and "concurrent rendering"?* — Batching = collect updates into one render. Concurrent rendering = pause/resume that render based on priority. Different mechanisms.

TOPIC 6 — STATE MANAGEMENT

Q1: Context vs Redux vs Zustand — when to use which

The honest framing for 2026:

Need	Reach for
Theme / locale / current user	Context
Server data (lists, queries)	React Query / TanStack Query — not a state library, a server-state cache
Small global UI state (modal open, sidebar)	Zustand or Jotai
Complex domain state with audit/time-travel/middleware	Redux Toolkit
Brand-new project, no specific need	Start with <code>useState</code> + Context. Add Zustand only when prop-drilling or context-rerender pain shows up.

The biggest myth: Redux is dead. It's not — Redux Toolkit is genuinely good for large apps. But for the **median React app in 2026**, you don't need it. React Query covers 80% of what people used Redux for (caching API responses).

- 1KB, no provider needed, hooks-first.
- Selector-based reads → no re-render unless your selected slice changes.
- Zero boilerplate compared to Redux.

```
import { create } from 'zustand';
const useStore = create(set => ({
  count: 0,
  inc: () => set(s => ({ count: s.count + 1 })),
}));

function Counter() {
  const count = useStore(s => s.count);
  const inc = useStore(s => s.inc);
  return <button onClick={inc}>{count}</button>;
}
```

- Every consumer re-renders when the value reference changes.
- No selector mechanism.
- Has to be at top of tree → forces every page to wrap in providers.
- Reaching for Redux for a 5-page CRUD app. Massive over-engineering.
- Storing server data in Redux/Zustand and reinventing caching/invalidation. Use React Query.
- Putting **everything** in Context until rerenders kill perf.
- *What does Redux Toolkit add over plain Redux?* — `createSlice` (auto-generated reducers + action creators), `createAsyncThunk`, RTK Query (built-in server-state). Plain Redux is rarely

written by hand anymore.

- *Why is "server state" different from "client state"? —* Server state is **shared**, **stale**, and needs **invalidation/refetch**. Client state (modal open, form draft) doesn't. They have different problems → different tools.

Q2: React Query / TanStack Query — server state done right

A: React Query (now **TanStack Query**) is a **server-state cache**. You give it a query key + a fetch function, it gives you data + loading/error states + caching + retries + dedup + stale-while-revalidate + automatic refetch on focus.

```
import { useQuery } from '@tanstack/react-query';

function User({ id }) {
  const { data, isLoading, error } = useQuery({
    queryKey: ['user', id],
    queryFn: () => fetch(`/api/users/${id}`).then(r => r.json()),
  });
  if (isLoading) return 'Loading...';
  if (error) return 'Error';
  return <p>{data.name}</p>;
}
```

- **Dedup:** 5 components mount and call the same query → 1 network request.
- **Cache:** revisit the page → shows cached data instantly, refetches in background.
- **Race conditions:** automatic — React Query keys responses to query keys.
- **Retries with backoff** on failure.
- **Pagination, infinite scroll, optimistic updates** — first-class APIs.
- **Mutations** with `useMutation` and cache invalidation:

```
const qc = useQueryClient();
const m = useMutation({
  mutationFn: data => fetch('/api', { method: 'POST', body: JSON.stringify(data) }),
  onSuccess: () => qc.invalidateQueries({ queryKey: ['users'] }),
});
m.mutate({ name: 'Rutik' });
```

- Using `useEffect` + `useState` for server data. You'll reinvent every problem React Query already solved, badly.
- Putting React Query results into Redux. Don't — it's already a cache.

- Wrong query keys. The key is the **identity** of the data; if it's wrong, cache hits/misses break.
- *SWR vs React Query?* — Same idea, smaller API. React Query has more features (mutations, infinite queries, devtools). SWR is leaner.
- *Where does this run on the server (RSC/Next.js)?* — Server Components fetch directly; React Query is for client-side data fetching. With Next.js, you can prefetch on the server and hydrate.

TOPIC 7 — REACT 18 / 19 FEATURES

Q1: Concurrent rendering & useTransition

A: React 18 made the **renderer concurrent**. Updates can be **paused, resumed, abandoned, or assigned priorities**. The user-facing feature that exposes this is **useTransition**.

```
function Search() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  function onChange(e) {
    setQuery(e.target.value); // urgent - user is typing
    startTransition(() => {
      setResults(filterBigList(e.target.value)); // non-urgent - can be interrupted
    });
  }

  return (
    <div>
      <input value={query} onChange={onChange} />
      {isPending && <Spinner />}
      <List items={results} />
    </div>
  );
}
```

What's happening: the input update has high priority and renders immediately. The expensive list filter runs **inside a transition** — React works on it in the background. If the user types again, React **abandons** the partial work and starts over with the new value. Result: input never lags.

Pre-18, the only way to keep the input snappy was **setTimeout(..., 0)** or a debounce — both worse.


```
const deferred = useDeferredValue(query);
// expensive list reads `deferred`, not `query` - falls behind during fast typing
```

- Wrapping a `setState` that the user **needs to see immediately** (like the input value itself) in a transition. The input becomes laggy. Only wrap derived/heavy work.
- Expecting `useTransition` to make slow code fast. It doesn't — it just makes it non-blocking. The work still runs.
- *Does this require any opt-in?* — `createRoot()` from `react-dom/client` enables concurrent features. Legacy `ReactDOM.render` doesn't.
- *How is this different from debounce?* — Debounce delays the work. `useTransition` runs the work but yields to higher-priority updates. The user gets fresher results faster.

Q2: Suspense for data fetching

A: `<Suspense fallback={...}>` lets a component **suspend** while it waits for something (lazy code, async data) and shows a fallback in the meantime. The component "throws a promise"; React catches it, renders the fallback, and re-renders the component when the promise resolves.

```
<Suspense fallback={<Spinner />}>
  <Profile userId={1} />
</Suspense>
```

Originally Suspense was only for `React.lazy` (code-splitting). Since 18+, with frameworks (Next.js, Remix) and libraries that integrate (React Query has a `suspense: true` mode, Relay, Apollo), it works for **data fetching** too.

lets you write:

```
function Profile({ userId }) {
  const user = useUser(userId); // suspends if data not ready
  return <h1>{user.name}</h1>;
}
```

...instead of `if (loading) return ...; if (error) return ...;` boilerplate everywhere.

Streaming SSR (React 18+): Suspense lets the server **flush HTML in chunks**. Above-the-fold ships first, slow data streams in below. Users see content fast.

- Trying to use raw `fetch()` inside a Suspense-aware component. You need a library that integrates (React Query w/ `suspense: true`, or a framework like Next.js).
- Treating `<Suspense>` as a global loader. It only catches **suspended descendants**. Place it close to the component that suspends.
- *Suspense vs error boundary?* — Suspense catches **promises** (loading state). Error boundaries catch **errors** (failure state). You typically wrap with both.
- *What's "Suspense waterfalls"?* — When child components each suspend on their own request → sequential network calls. Fix: **prefetch in the parent** or use a router with parallel data loading.

Q3: React Server Components (RSC)

A: Server Components run **only on the server**, never ship to the client, and can directly access databases / file systems / secrets. They render to a special **streaming format** that the client merges with **Client Components** (anything with `'use client'` at the top).

The mental model:

- **Server Component** — async, fetches data, no hooks, no event handlers, no `useState`. Zero JS shipped.
- **Client Component** — what you've always written. Interactive. Ships to the browser.

```
// app/page.tsx — Server Component (default in Next.js App Router)
async function Page() {
  const posts = await db.posts.findMany();
  return <PostList posts={posts} />;
}

// post-list.tsx
'use client';
export function PostList({ posts }) {
  const [filter, setFilter] = useState('');
  // ...interactive, runs in the browser
}
```

- **Smaller bundles** — non-interactive UI is rendered once on the server, never shipped.
- **Direct data access** — no `/api/...` boilerplate just to read from the DB.
- **Secrets stay server-side** — API keys, DB connections, can be referenced in server components without leaking.

anything that uses state, effects, browser APIs, or event handlers must be in a client component. RSC pushes you to keep interactivity at the leaves.

- Using `useState` / `useEffect` in a server component → build error.
- Importing a server component **into** a client component (the other way is fine).
- Forgetting that **props passed from server to client must be serializable** (no functions, no Dates with methods, no class instances).
- *Where do RSCs run today?* — **Next.js App Router** is the production-ready home. Vanilla React doesn't support them yet.
- *RSC vs traditional SSR?* — SSR renders the **whole page** as HTML on every request. RSC renders only the server parts and **streams a special payload** the client incrementally hydrates. Smaller payload, finer-grained.

TOPIC 8 — FORMS & ROUTING

Q1: React Hook Form — why everyone uses it

A: Most form libraries make every keystroke re-render the entire form. **React Hook Form** uses **uncontrolled inputs + refs** under the hood, so typing in one field doesn't re-render the others. On a 50-field form this is the difference between buttery and laggy.

```
import { useForm } from 'react-hook-form';

function SignUp() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  return (
    <form onSubmit={handleSubmit(d => console.log(d))}>
      <input {...register('email', { required: 'Email is required' })} />
      {errors.email && <span>{errors.email.message}</span>}

      <input {...register('password', { minLength: 8 })} />
      <button>Submit</button>
    </form>
  );
}
```

- Field-level validation with messages.
- Schema validation via Zod / Yup integration.
- Dirty/touched tracking.
- **Controller** wrapper for working with controlled UI components (e.g. MUI inputs, custom date pickers).

- Tiny bundle, zero deps.
- Using controlled state (`useState` per field) "to be safe." Negates the perf benefit.
- Forgetting that `register('field')` returns `ref+name+onChange` — you spread it onto the input, not assign it.
- *RHF vs Formik?* — Formik is controlled-by-default (re-renders heavy). RHF is uncontrolled. RHF wins on perf and bundle size; Formik's API is sometimes simpler.
- *How do you integrate with Zod?* — `@hookform/resolvers/zod` → `useForm({ resolver: zodResolver(schema) })`. Type-safe, single source of truth for validation.

Q2: React Router v6 — loaders & actions

A: React Router is the de-facto routing library. v6 (and v6.4+) introduced **data routers** — routes can declare a `Loader` that runs **before** the component renders, eliminating the "render → effect → fetch → loading flash" waterfall.

```
import { createBrowserRouter, RouterProvider } from 'react-router-dom';

const router = createBrowserRouter([
  {
    path: '/users/:id',
    loader: async ({ params }) => {
      const r = await fetch(`/api/users/${params.id}`);
      return r.json();
    },
    Component: UserPage,
  },
]);

function UserPage() {
  const user = useLoaderData(); // already fetched
  return <h1>{user.name}</h1>;
}

function App() { return <RouterProvider router={router} />; }
```

Why this matters: the route renders with data **already present**. No `useEffect` + `useState` + loading UI per page.

— it runs on form submission and can revalidate the page after.

```

{
  path: '/users/:id/edit',
  action: async ({ request, params }) => {
    const fd = await request.formData();
    await fetch(`/api/users/${params.id}`, { method: 'PUT', body: fd });
    return redirect(`/users/${params.id}`);
  },
}

```

- Mixing data routers with the legacy `<BrowserRouter>` API. Pick one — `createBrowserRouter` is the modern path.
- Doing data fetches inside the component anyway, defeating the loader.
- *Why is this similar to Remix?* — Because Remix's API was the inspiration. React Router v6.4+ is essentially "Remix routing without the server framework."
- *Can a loader stream?* — Yes — `defer({ data: slowPromise })` + `<Await>` lets the page render with fast data while slow data streams.

TOPIC 9 — TESTING & PATTERNS

Q1: React Testing Library — guiding principles

A: RTL's mantra: **"Test what the user sees, not how the component is implemented."** No shallow rendering, no testing of internal state — query by accessible name, simulate real interactions.

```

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

test('counter increments on click', async () => {
  render(<Counter />);
  const btn = screen.getByRole('button', { name: /increment/i });
  await userEvent.click(btn);
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});

```

1. `getByRole` — what assistive tech sees. Best.
2. `getByLabelText` — for form fields.
3. `getByPlaceholderText` — when no label exists.

4. `getByText` — for non-interactive text.
5. `getByDisplayValue` — for prefilled inputs.
6. `getByAltText` — for images.
7. `getByTitle` — last resort.
8. `getByTestId` — escape hatch. Only if nothing semantic works.

Why RTL replaced Enzyme: Enzyme tested implementation details (`wrapper.state()`, `wrapper.instance()`). When you refactored, tests broke even when behavior was unchanged. RTL tests survive refactors because they only see what users see.

- Using `getByTestId` everywhere. Lazy and fragile.
- `act()` warnings — usually means you forgot `await` on a `userEvent` interaction or a state update happened outside React's awareness.
- Mocking `useState` / hooks. Don't — render the component normally.
- What's `findBy*` vs `getBy*`? — `getBy*` throws immediately if not found. `findBy*` returns a promise that retries until the element appears (for async content). `queryBy*` returns null instead of throwing — use only when asserting absence.
- *Vitest vs Jest?* — Vitest is a faster, ESM-native runner. Same APIs as Jest. Default in Vite projects.

Q2: HOC vs Render Props vs Compound Components

Three pre-hooks patterns for sharing logic between components. Hooks replaced 90% of HOC and render-prop use cases. Compound components are still common.

— function that takes a component and returns a new one with extra behavior.

```
const withLogger = (Wrapped) => (props) => {
  useEffect(() => console.log('mount'), []);
  return <Wrapped {...props} />;
};

const ProfileWithLogger = withLogger(Profile);
```

Pre-hooks, used for `connect()` (Redux), `withRouter`, etc. Hooks killed it because hooks compose better and don't have the "wrapper hell" problem.

— pass a function as a child / prop that returns JSX.

```
<Mouse render={({ x, y }) => <p>{x},{y}</p> />
```

Replaced by custom hooks for the same reason.

— set of components designed to work together with implicit shared state via Context.

```
<Tabs defaultValue="a">
  <Tabs.List>
    <Tabs.Trigger value="a">A</Tabs.Trigger>
    <Tabs.Trigger value="b">B</Tabs.Trigger>
  </Tabs.List>
  <Tabs.Panel value="a">A content</Tabs.Panel>
  <Tabs.Panel value="b">B content</Tabs.Panel>
</Tabs>
```

This is the dominant pattern in modern UI libraries (Radix, shadcn/ui, Headless UI). Flexible, composable, no prop-drilling.

- Reaching for HOCs in 2026. Use a custom hook unless you specifically need to wrap the component itself.
- Building a compound component without exposing the internal Context — children can't read state, defeating the pattern.
- *Why is "wrapper hell" a problem with HOCs?* — `withA(withB(withC(withD(Component))))`. Stack traces are unreadable, debugging is a nightmare, types fight you.
- *When IS an HOC still the right answer?* — When you need to wrap **arbitrary unknown components** at boundaries (e.g., `withErrorBoundary`, `withSentry`).

TOPIC 10 — REAL-WORLD & TRICKY

Q1: Why should keys NOT be the array index?

A: Already touched in Topic 5 Q1, but this is the **single most asked React interview question** so worth standalone treatment.

The key tells React: "this element corresponds to *that* element from the previous render — preserve its DOM node, its component state, its focus, its scroll position."

When you use the as the key:

- Insert at front → every existing item now has a new index → React thinks **every item changed** and remounts the whole list.
- Delete from middle → items below shift down → their state belongs to the **wrong items** now.

- Reorder → state and DOM stick to **positions**, not items.

```
function Todos({ todos }) {
  return todos.map((t, i) => (
    <li key={i}>
      <input defaultValue={t.text} /> // uncontrolled
    </li>
  ));
}
```

User types "a" into the second `<input>`, then deletes the first `<li>`. Result: "a" is now in the **first** `<li>`, because the DOM node at index 1 became the DOM node at index 0. **Wrong**.

Fix: `key={t.id}`. Now React matches by ID — when item 0 is removed, the item-1 node is moved to position 0 with its state intact, but it's the **right** item.

- Using `key={Math.random()}` or `key={Date.now()}`. Now nothing matches → every render is a full remount → animations break, focus is lost, state resets.
- "But my list never changes" — if it never changes, you don't render it dynamically anyway. If you do render it dynamically, give it real keys.
- *When is index-as-key acceptable?* — When the list is static, never reorders or filters, and items have no internal state. Pure read-only lists.
- *What if my data has no ID?* — Generate one (`crypto.randomUUID()`) when you create the item, not when you render it.

Q2: Memory leaks in useEffect

A: A memory leak in React almost always comes from **a subscription that outlives the component**. Cleanup is the answer.

The big four sources:

```
useEffect(() => {
  function onKey(e) { /* ... */ }
  window.addEventListener('keydown', onKey);
  return () => window.removeEventListener('keydown', onKey); // ✅ required
}, []);
```

4. State updates after unmount. When a slow fetch resolves on a component that already unmounted, the `setState` call triggers a warning and (sometimes) a leak via the captured closure.


```
// ✗ leaks if component unmounts before fetch resolves
useEffect(() => {
  fetch('/x').then(r => r.json()).then(setData);
}, []);

// ✓ cancel-flag pattern (see Topic 2 Q3)
useEffect(() => {
  let cancelled = false;
  fetch('/x').then(r => r.json()).then(d => { if (!cancelled) setData(d); });
  return () => { cancelled = true; };
}, []);
```

is React telling you exactly this. It was removed in 18 (because it was misleading more often than helpful), but the underlying issue remains.

- Adding a listener inside the component body, not in an effect. Runs on every render → multiple listeners stack up.
- Removing the listener with a different function reference. The function passed to `removeEventListener` must be the **same reference** as `addEventListener`.
- *How do I detect leaks in production?* — Memory profiler in Chrome DevTools — take a heap snapshot, navigate, take another, look for components that should have been collected.
- *Does Strict Mode help?* — Yes — it double-invokes effects + cleanups, surfacing missing cleanups in development.

Q3: Error Boundaries — why they exist

A: A normal React error inside a render or lifecycle **crashes the whole tree** — all your UI disappears and you see a blank page (or the dev overlay). **Error boundaries** are class components (still — there's no hook equivalent yet) that **catch** errors from descendants and render a fallback UI.

```
class ErrorBoundary extends React.Component {
  state = { error: null };
  static getDerivedStateFromError(error) { return { error }; }
  componentDidCatch(error, info) { logToSentry(error, info); }
  render() {
    if (this.state.error) return <p>Something broke.</p>;
    return this.props.children;
  }
}

// Usage
<ErrorBoundary>
  <RiskyChart />
</ErrorBoundary>
```

errors thrown in render, lifecycle, constructors of descendants.

- Event handlers (`onClick`) — wrap in try/catch yourself.
- Async code (promises, `setTimeout`) — same.
- Errors in the boundary itself.
- SSR errors (caught at server level).

Real-world placement: wrap each route, plus any **risky widget** (3rd-party chart, embedded iframe), with its own boundary. Don't put one boundary at the root and call it a day — then any error still kills your whole app.

- One boundary at the top of the app. Everything looks fine until any leaf throws → entire app blank.
- Forgetting that event handlers don't bubble up to boundaries.
- No reset mechanism — once a boundary errors, it stays errored. Pass a `resetKeys` prop or use `react-error-boundary` library.
- *Why no hook equivalent?* — Error boundaries are tied to React's class lifecycle (`getDerivedStateFromError`). Hooks-based error boundaries are an active proposal.
- *Library most teams use?* — `react-error-boundary` (by Kent C. Dodds) — wraps the boilerplate, adds reset logic.

Q4: Lifting state up vs colocation

Two opposite-direction principles, both correct in the right context.

Lift state up when **two siblings need the same state**.

```

// ✗ Each input owns its own state — they can't sync
function Form() {
  return <<FirstNameInput /><FullNameLabel /></>;
}

// ✓ Lift to common parent
function Form() {
  const [first, setFirst] = useState('');
  return <<FirstNameInput value={first} onChange={setFirst} /><FullNameLabel first={first} /></>;
}

```

Colocate state when **only one component uses it**. Don't lift unnecessarily — it pollutes the parent and forces re-renders down the whole tree.

```

// ✗ Lifted unnecessarily — parent re-renders on every keystroke
function App() {
  const [draft, setDraft] = useState('');
  return <<CommentBox draft={draft} setDraft={setDraft} /><HugeChart /></>;
}

// ✓ Colocated — only CommentBox re-renders
function App() {
  return <<CommentBox /><HugeChart /></>;
}

function CommentBox() {
  const [draft, setDraft] = useState('');
  return <textarea value={draft} onChange={e => setDraft(e.target.value)} />;
}

```

The rule: state lives at the **lowest common ancestor** of components that read or write it. Not higher.

- Lifting all state to the top "for consistency." Causes giant re-render trees.
- Refusing to lift when two components clearly need the same value, then syncing via effects (always wrong).
- *What if two unrelated components need the same state?* — Lift to a common ancestor (might be the route or app root) — or use Context / Zustand for true cross-tree state.
- *How do you decide where to colocate?* — Start at the **lowest possible level** (in the component using it). Lift only when a sibling needs it too.

CLOSING

You made it through the roadmap. A few parting truths from real interviews:

1. **Hooks rules** trip up ~30% of candidates. If you can confidently explain *why* hooks must be called in the same order every render, you're already in the top half.
2. **Reconciliation + keys** is the #1 most-asked React question. Memorize it.
3. **Performance questions** are usually tests of restraint. Saying "I'd profile first before memoizing" is almost always the right answer.
4. **React 18+ features** (Concurrent, Suspense, RSC) — even fresher roles ask about these in 2026. Don't skip them.

Good luck — and ship the boring solution before the clever one.

— Rutik